

Sesjon 2 — Oppgaver

Blokk 1: Sources og seeds

OPPGAVER

1. Utforsk kildedataene

Åpne BigQuery og ta en titt på tabellene i `raw`-datasettet.

Legg merke til kolonnenavn, datatyper og et par rader med data.

Spørsmål å tenke på:

- Hvilke kolonner ser rare ut?
- Hva mangler?

2. Registrer kildene i YAML

Opprett filen `models/sources.yml` og registrer alle fire kildesystemer.

Krav:

- Bruk meningsfulle `name`-verdier for hver source-gruppe
- Sett `schema: raw` på alle
- Inkluder alle 7 tabeller

Sjekk at dbt leser konfigurasjonen uten feil:

```
dbt parse
```

3. Legg til freshness på CRM-kildene

`crm_customers` og `crm_contracts` har kolonnen `created_at` .

- Sett `loaded_at_field: created_at` på CRM-sourcen
- Definer fornuftige `warn_after` og `error_after` -verdier
- Kjør `dbt source freshness` og se hva som skjer

Hva rapporterer dbt? Er resultatet forventet gitt datoene i dataene?

4. Legg til beskrivelser

Legg til en `description:` på hver kildetabell i YAML.

Hold det kort — én setning som forklarer hva tabellen inneholder og hvilket system den kommer fra.

5. Freshness per tabell

Ulike tabeller oppdateres med ulik frekvens.

Definer individuelle freshness-terskler per tabell i stedet for én felles per source.

Tenk igjennom:

- `billing_invoices` vs. `catalog_products` — bør de ha samme terskel?
- Hva er konsekvensen av en for streng terskel på en tabell som bare oppdateres månedlig?

6. `meta`-felter

dbt lar deg legge til vilkårlige nøkkelverdi-par under `meta:` på en source eller tabell.

```
tables:  
  - name: crm_customers  
    meta:  
      owner: crm-team  
      update_frequency: daily  
      pii: true
```

Legg til relevante `meta`-felter på minst to av kildene.

Disse er ikke funksjonelle i seg selv, men dukker opp i docs-siden og kan leses av verktøy.

DISKUSJON

A. Struktur på source-filer

Bør alle kildene samles i én `__sources.yml`, eller én fil per kildesystem (`_crm.yml`, `_billing.yml`)? Hva er fordeler og ulemper med begge? Hva styrer valget i en stor organisasjon med mange kildesystemer?

B. Eierskap av freshness-terskler

Hvem bør bestemme freshness-terskler — datateamet, kildesystemets eiere, eller forretningssiden? Hva skjer når en terskel utløses i produksjon kl. 03:00 en søndag? Og hva skjer i helger og helligdager når batch-jobber ikke kjører?

C. Seed vs. kilde

Produktkatalogen er lastet som en seed her, men den kunne like gjerne kommet fra et kildesystem. Hva er kriteriene for å velge seed fremfor source? Når begynner en seed å bli feil løsning?

D. Dokumentasjon som kode

Beskrivelser i YAML er kode som kan versjonskontrolleres og gjennomgås i pull requests. Hva er den praktiske utfordringen med å holde dem oppdatert over tid? Finnes det måter å håndheve dokumentasjonsdekning på?

EKSTRAOPPGAVER

E1. YAML-ankere

`__sources.yml` kan ende opp med gjentatt freshness-konfigurasjon på tvers av tabeller. YAML støtter ankere (`&`) og referanser (`*`) for å definere verdier én gang og gjenbruke dem:

```
_freshness_daily: &freshness_daily
  warn_after: {count: 24, period: hour}
  error_after: {count: 48, period: hour}

tables:
  - name: crm_customers
    freshness: *freshness_daily
  - name: crm_contracts
    freshness: *freshness_daily
```

Definer én eller to freshness-profiler og bruk dem på tvers av relevante tabeller. Kjør `dbt parse` og `dbt source freshness` og bekreft at det fortsatt fungerer.

YAML-ankere er standard YAML — ikke en dbt-funksjon. De fungerer overalt YAML brukes.

E2. Singulære tester

En singulær test er en SQL-fil i `data-tests/` som returnerer rader som feiler testen. dbt feiler testen hvis én eller flere rader returneres. Det er den enkleste testformen i dbt — ingen makroer, ingen konfigurasjon, bare SQL.

Se «Singular data tests» i [dbt-dokumentasjonen](#).

Identifiser én forretningsregel i kildedataene som ikke dekkes av de eksisterende generiske testene (`unique` , `not_null` , `accepted_values` , `relationships`). Skriv en singulær test for den.

Noen kandidater å vurdere:

- Datakolonner som bør ha en logisk rekkefølge
- Beløp som har en forventet øvre eller nedre grense
- Referanser som bør finnes i begge retninger

```
dbt test --select <modell_eller_kilde>
```

Diskuter: Når passer singulære tester bedre enn generiske? Hva er ulempen med singulære tester i en stor kodebase?

Blokk 2: Staging-modeller og materialisering

OPPGAVER

1. Bygg `stg_crm__customers`

Opprett `models/staging/stg_crm__customers.sql`.

Krav:

- Rename `custID` → `customer_id`
- Cast `created_at` til `DATE`
- Behold `address` og `preferences` som strenger (JSON pakkes ut senere)
- Referer til kilden med `{{ source('crm', 'crm_customers') }}`

Registrer modellen i `models/staging/_staging.yml` med `materialized: view`.

```
dbt run --select stg_crm__customers
```

2. Bygg `stg_billing_invoices`

`billing_invoices` har tre kolonner som trenger behandling.

Krav:

- Rename `InvoiceID` → `invoice_id`
- Cast `invoice_date` og `due_date` fra `YYYYMMDD`-streng til `DATE`
- Cast `amount_eur` fra `STRING` til `NUMERIC`

Tips for datokonvertering i BigQuery:

```
parse_date('%Y%m%d', invoice_date) as invoice_date
```

Hva skjer med de krediterte radene der `amount_eur` er negativ etter casten? Er det et problem her, eller hører det hjemme i en test?

3. Registrer alle staging-modeller i YAML

Opprett eller utvid `models/staging/_staging.yml` med alle 7 staging-modeller.

Krav:

- `materialized: view` på alle
- Minst én `description:` per modell
- Kjør alle modellene og verifiser at de kompilerer:

```
dbt run --select staging
```

4. Bygg de resterende staging-modellene

Bygg `stg_crm__contracts`, `stg_billing__payments`, `stg_network__incidents` og `stg_catalog__products`.

Merk:

- `stg_catalog__products` skal bruke `{{ ref('catalog_products') }}`, ikke `{{ source() }}` — hvorfor?
- `network_incidents` bruker `customer_ref` som fremmednøkkel. Skal du rename den til `customer_id`?

5. Materialisering på mappenivå

Flytt materialiseringskonfigurasjonen ut av enkeltmodellenes YAML og inn i `dbt_project.yml` som en mappenivå-setting.

```
models:
  mitt_prosjekt:
    staging:
      +materialized: view
```

- Fjern `materialized:` fra hver enkelt modell i `_staging.yml`
- Kjør `dbt run --select staging` og bekreft at ingenting endret seg

Hva er fordelen med å sette materialisering sentralt fremfor per modell? Når vil du overstyre det per modell?

6. JSON i staging

`crm_customers` har to JSON-kolonner: `address` (flat adressestruktur) og `preferences` (kommunikasjonspreferanser).

Pakk ut `address`-feltet i `stg_crm__customers`:

```
json_value(address, '$.street') as address_street,  
json_value(address, '$.city')   as address_city,  
json_value(address, '$.zip')    as address_zip,  
json_value(address, '$.country') as address_country,
```

Diskuter deretter:

- Er det riktig å pakke ut `address` i staging? Hva med `preferences`?
- Hva er argumentet for å vente til intermediate?
- Hva vil du gjøre med rader der `preferences` er `{}`?

DISKUSJON

A. Navnekonvensjon i staging

`stg__crm_customers` bruker dobbelt understrek mellom lag og navn. Hva er prinsippet bak dette? Bør kildesystemnavnet (`crm`) alltid inkluderes i modellnavnet, eller er det kontekst som heller hører hjemme i mappestrukturen? Hva er praksisen i din organisasjon?

B. Én staging-modell per kildetabell — alltid?

Konvensjonen sier én staging-modell per kildetabell. Finnes det legitime unntak? Hva om to kildetabeller inneholder identisk format fra to regioner og aldri brukes separat? Hva er risikoen ved å slå dem sammen i staging?

C. JSON-utpakking: staging eller intermediate?

Vi pakket ut `address` i staging, men `device_info` ble utsatt til intermediate. Hva er prinsippet? Er det et klart skille, eller er det et skjønnsspørsmål? Hva gjør dere hvis teamet er uenige om plasseringen?

D. Testing av kjente feil

Vi vet at noen kunder mangler e-post, og at `PP_L` er duplikat i produktkatalogen. Bør vi skrive tester som vi vet vil feile? Hva er verdien av det? Hva er alternativet — og hva er risikoen ved å ikke teste dem?

EKSTRAOPPGAVER

E3. Test-alvorlighetsgrad

Ikke alle testfeil er like alvorlige. dbt lar deg sette `severity: warn` på en test for å rapportere et varsel uten å feile pipelinen.

Se `severity`-konfigen i [dbt-dokumentasjonen](#).

Gå gjennom testene i `_staging.yml` og klassifiser dem:

- `severity: error` — testfeil betyr at downstream-modeller ikke kan stoles på
- `severity: warn` — testfeil dokumenterer et kjent kildesystemproblem som ikke stopper pipelinen

```
- name: invoice_ref
  tests:
    - unique:
        config:
          severity: warn
```

Oppdater alle relevante tester med riktig alvorlighetsgrad og observer forskjellen i `dbt test`-output.

E4. `store_failures`

Når en test feiler er det nyttig å se hvilke rader som feiler, ikke bare at testen feiler. `store_failures: true` ber dbt materialisere de feilende radene som en tabell i databasen.

Se `store_failures`-konfigen i [dbt-dokumentasjonen](#).

Legg til `store_failures` på én av testene som forventes å feile. Kjør `dbt test` og finn tabellen dbt opprettet. Hva inneholder den, og hvor i databasen ligger den?

Tips: `store_failures: true` kan også settes globalt i `dbt_project.yml` for alle tester.

E5. Generiske tester

dbt støtter to typer tester: *singulære* (én SQL-fil per test) og *generiske* (Jinja-makroer definert én gang i `macros/` som kan brukes på vilkårlige kolonner via YAML-konfigurasjon).

En generisk test:

- Defineres som en makro med signaturen `test_<navn>(model, column_name)`
- Returnerer rader som feiler testen
- Brukes i YAML på samme måte som de innebygde testene (`unique`, `not_null`, osv.)

Se «Custom generic tests» i [dbt-dokumentasjonen](#).

Skriv en generisk test som er nyttig for kolonner i dette prosjektet. Bruk den på minst to kolonner i `_staging.yml` eller `_intermediate.yml`.

Tenk igjennom: Hva er fordelen med en generisk test fremfor en singulær test for samme sjekk? Og hva skiller dette fra `dbt_utils.expression_is_true`?

E6. Doc-blokker

`customer_id` forekommer i nesten alle modeller, men beskrivelsen gjentas i dag i hver YAML-fil. dbt støtter gjenbrukbare beskrivelser via doc-blokker definert i `.md`-filer.

Se «Docs blocks» i [dbt-dokumentasjonen](#).

Opprett `models/docs.md` og definer doc-blokker for minst to kolonner som går igjen på tvers av modeller. Erstatt de gjentatte beskrivelsene i YAML-filene med referanser til doc-blokkene.

Generer docs og verifiser at beskrivelsene vises korrekt:

```
dbt docs generate && dbt docs serve
```

Blokk 3: Intermediate-modeller

OPPGAVER

1. Bygg `int_customers__enriched`

Opprett `models/intermediate/int_customers__enriched.sql`.

Krav:

- Join `stg_crm__customers` med `stg_crm__contracts`
- Behold kun **siste aktive kontrakt** per kunde (hint: `ROW_NUMBER()` over `start_date`)
- Pakk ut `preferences` -JSON: `preferred_language`, `paperless_billing`, `marketing_opt_in`
- Håndter kunder uten noen kontrakt (C017 har ingen)

Registrer modellen i `models/intermediate/_intermediate.yml` med `materialized: table` og minst tre kolonnenivå-beskrivelser.

```
dbt run --select int_customers__enriched
```

2. Bygg `int_billing__invoice_settlement`

Her møter vi en reell designutfordring: faktura og betaling er to separate entiteter — men vi skal likevel koble dem i intermediate. Er det i tråd med prinsippet om at intermediate handler om én entitet?

To gyldige tilnærminger:

Tilnærming A — faktura som primærentitet (det vi gjør her):

Modellen har én rad per faktura. Betalingen er ikke en likeverdig entitet — den er et oppslag som svarer på spørsmålet *er denne fakturaen gjort opp?* Joinen tjener fakturaen. Modellnavnet `int_billing__invoice_settlement` signaliserer dette tydelig.

Tilnærming B — hold dem adskilt:

`int_billing__invoices` og `int_billing__payments_deduped` er separate modeller. Joinen skjer først i mart, der konsumenten faktisk trenger begge. Dette er renere i teorien, men betyr at dedupliseringslogikken for betalinger ikke er gjenbrukbar uten en egen modell for det.

Begge er forsvarbare. Valget avhenger av om deduplisering av betalinger er noe mange marts vil trenge — i så fall løfter du det opp. Hvis bare én mart noensinne bruker det, kan det like gjerne leve der.

Implementer tilnærming A:

Opprett `models/intermediate/int_billing__invoice_settlement.sql`.

Krav:

3. Legg til tags og kolonnenivå-beskrivelser

Utvid `_intermediate.yml` med:

- Tag `customer` på alle modeller som handler om kunder
- Tag `billing` på faktura- og betalingsmodellene
- Fullstendige kolonnenivå-beskrivelser på alle kolonner i `int_customers__enriched`

Verifiser at tags fungerer:

```
dbt run --select tag:customer
```


4. Bygg `int_customers__activations`

Koble `stg_crm__customers` med `stg_network__service_activations`.

Utfordringer å håndtere:

- Noen kunder har to aktiveringsrader (oppgradering av enhet) — beholder du den nyeste?
- `device_info` inneholder null-verdier for `imei` og `os` — håndter dette eksplisitt
- Pakk ut alle fire device-felter og cast `os` til lowercase for konsistens

5. Bygg `int_network__incidents_enriched`

Koble `stg_network__incidents` med `int_customers__enriched` for å berike hendelsene med kundeinformasjon.

Krav:

- Join på `customer_ref` → `customer_id` (merk: ulike kolonnenavn i de to modellene)
- Inkluder `customer_name`, `status` og `contract_status` fra kundemodellen
- Beregn `resolution_time_minutes` for løste hendelser (`resolved_at is not null`)
- Behold uløste hendelser i resultatet — ikke filtrer dem bort

Legg til modellen i `_intermediate.yml` med tag `network` og kolonnenivå-beskrivelser.

6. Intermediate som gjenbrukbart lag

`int_customers__enriched` er nå bygget. Se for deg at du skal lage to ulike marts:

- `mart_customer_360` — én rad per kunde med alle attributter
- `mart_churn_candidates` — kunder med suspendert status og åpne hendelser

Spørsmål:

- Hvilken logikk ville blitt duplisert i begge marts hvis intermediate ikke fantes?
- Er det noe i `int_customers__enriched` som egentlig er konsumentspesifikt og burde vært i mart i stedet?
- Hvor går grensen i akkurat dette tilfellet?

DISKUSJON

A. Intermediate vs. mart — hvor går grensen?

Intermediate inneholder logikk som er universell for entiteten, mart inneholder konsumentspesifikk logikk. Men grensen er ikke alltid klar. Ta `is_paid`-flagget i `int_billing__invoice_settlement`: er det universelt (en faktura er enten betalt eller ikke) eller konsumentspesifikt (bare relevant for churn-rapporter)? Diskuter tre–fire kolonner fra modellene dere har bygget og klassifiser dem.

B. Å eksponere datakvalitetsproblemer

`int__billing_invoice_settlement` fjerner stille dupliserte betalinger ved å beholde den første. En alternativ tilnærming er å eksponere et `is_duplicate`-flagg og la mart-laget bestemme hva som skal skje. Hva er fordeler og ulemper? Hvem bør vite om kildeproblemet — bare datateamet, eller også analytikerne som bruker mart-tabellene?

C. Navnekonvensjon i intermediate

`int__customers_enriched` og `int__customers_activations` bruker entiteten (`customers`) som primær gruppe. Alternativet er å bruke kilden: `int__crm_customers`, `int__network_activations`. Hva er forskjellen i praksis når prosjektet vokser? Hva signaliserer navnevalget om eierskap og ansvar?

D. Dokumentasjon av designvalg

Intermediate-modellene inneholder viktige beslutninger: «vi beholder første betaling ved duplikat», «C017 har ingen kontrakt og beholder null-verdier». Hvor hører disse beslutningene hjemme — i YAML-beskrivelsen, i en SQL-kommentar, i PR-beskrivelsen, eller et annet sted? Hva er holdbart over tid når koden endrer seg?

EKSTRAOPPGAVER

E7. `dbt_utils`-tester

`dbt_utils`-pakken er allerede installert og gir tilgang til en rekke ekstra testtyper utover de innebygde. Se alle tilgjengelige tester i [dbt-utils-dokumentasjonen](#).

Aktuelle tester å utforske for dette prosjektet:

```
- dbt_utils.expression_is_true:  
  expression: "<uttrykk som skal være sant>"  
- dbt_utils.not_empty_string  
- dbt_utils.at_least_one
```

Finn kolonner i intermediate-modellene der disse testene ville fange reelle problemer. Legg dem til i `_intermediate.yml` og kjør `dbt test`.

E8. Fullstendig dokumentasjonsdekning

Åpne `_intermediate.yml` og tell opp kolonner uten `description:`. Mål: alle kolonner i alle fire intermediate-modeller skal ha en beskrivelse.

For hver modell skal modellnivå-beskrivelsen svare på:

1. Hva er primærentiteten (én rad per hva)?
2. Hvilke kildemodeller joinest, og på hvilken nøkkel?
3. Hvilke datakvalitetsproblemer håndteres her?

Generer docs og inspiser resultatet:

```
dbt docs generate && dbt docs serve
```